

FICHA 4:

Enunciado prueba parcial 4

Listas, diccionarios, funciones

Sigla	Asignatura	Experiencia de Aprendizaje
FPY1101	Fundamentos de Programación	EA3: Colecciones y funciones en Python
Tiempo	Modalidad de Trabajo	Indicadores de logro
2 h	Individual	IL 3.1 al IL 4.2

Resolver

Desarrolla un programa en Python que implemente un sistema de gestión de vehículos de un concesionario, donde todo el comportamiento se organice mediante funciones bien definidas. El programa debe incluir un menú interactivo, validaciones de entrada, operaciones lógicas (decisiones y comparaciones) y uso de funciones separadas.

1. Datos que debe manejar el sistema

El sistema trabaja con una colección de vehículos. Esta colección debe existir desde que el programa inicia y estar disponible durante toda la ejecución. Cada vez que se agrega un vehículo, se incorpora a esa colección como un nuevo elemento.

Cada vehículo se representa como un conjunto de campos asociados: modelo, año, precio y un indicador de si está disponible o no. La siguiente tabla resume los campos de cada registro:

Campo	Qué representa	Restricciones de validación
"modelo"	Modelo del vehículo	No vacío ni solo espacios en blanco
"anio"	Año de fabricación del vehículo	Número entero mayor que 1900
"precio"	Precio de venta del vehículo en pesos	Número decimal mayor que cero
"disponible"	¿El vehículo está disponible para la venta?	False al registrar. No se valida ni se solicita al usuario: el sistema

		lo asigna automáticamente. Su valor puede cambiar a True cuando se ejecute la opción 4 (Actualizar disponibilidad), según el año del vehículo.
--	--	--

Cada diccionario se guarda dentro de una lista. La lista es la colección general; los diccionarios son los vehículos individuales dentro de ella. El programa comienza con la lista vacía y la va llenando a medida que se agregan registros.

2. Lo que debe hacer el sistema

El sistema se controla desde un menú que aparece en pantalla cada vez que el usuario termina una acción. El usuario elige una opción numérica, el programa ejecuta la tarea correspondiente y vuelve a mostrar el menú. Esto se repite hasta que el usuario elige salir.

El menú tiene seis opciones:

```
===== MENÚ PRINCIPAL =====
1. Agregar vehículo
2. Buscar vehículo
3. Eliminar vehículo
4. Actualizar disponibilidad
5. Mostrar vehículos
6. Salir
=====
```

Para implementar este comportamiento debes definir dos funciones separadas: una que muestre las opciones en pantalla (sin recibir nada ni retornar nada) y otra que lea y retorne la opción elegida por el usuario (sin recibir nada, retornando el número validado). Ambas funciones deben invocarse en cada vuelta del ciclo.

A continuación, se describe qué debe ocurrir al elegir cada opción:

Opción 1 - Agregar vehículo:

El sistema solicita al usuario el modelo, el año y el precio del vehículo. Antes de guardar el registro, verifica que cada dato cumpla su condición:

- El modelo no puede estar vacío ni ser solo espacios en blanco.
- El año debe ser un número entero mayor que 1900.
- El precio debe ser un número decimal mayor que cero.

Si algún dato no cumple la condición, el sistema informa al usuario y no registra el vehículo. Solo cuando todos los datos son válidos se crea el diccionario y se agrega a la lista.

Para implementar esta opción debes definir una función que reciba la lista como parámetro. Dentro de ella se solicitan los datos al usuario y se llama a una función de validación distinta para cada campo. Los mensajes de error se muestran en esta función, no dentro de las validaciones.

Opción 2 - Buscar vehículo:

El sistema solicita un modelo del vehículo al usuario y recorre la lista buscando un registro cuyo campo modelo coincida exactamente con el ingresado. Si lo encuentra, muestra la posición en la que está y sus datos. Si no existe ningún registro con ese modelo, informa al usuario.

Para implementar esta opción debes definir una función que reciba la lista y el modelo a buscar como parámetros. La función recorre la lista y retorna la posición del registro encontrado, o -1 si no existe. Es el programa principal quien recibe ese valor y decide qué hacer con él: si la posición es válida, muestra los datos del vehículo en esa posición; si es -1, muestra el mensaje de no encontrado.

Opción 3 - Eliminar vehículo:

El sistema solicita el modelo del vehículo que se desea eliminar. Para localizarlo, llama a la función de búsqueda definida en la opción anterior, pasándole la lista y el modelo ingresado. Si la función retorna una posición válida, el sistema elimina el registro en esa posición. Si retorna -1, informa al usuario con el siguiente mensaje:

El vehículo 'modelo' no se encuentra registrado.

Opción 4 - Actualizar disponibilidad:

El sistema recorre la lista completa de vehículos y actualiza el campo "disponible" de cada registro según su año: si el año es mayor o igual a 2020, el campo pasa a True; si es menor, queda en False. Esta operación afecta a todos los registros de la lista sin excepción.

Para implementar esta opción debes definir una función que reciba la lista como parámetro y aplique esa regla a cada elemento.

Opción 5 - Mostrar vehículos:

El sistema primero actualiza la disponibilidad de todos los vehículos haciendo el llamado a la función anterior, luego recorre la lista mostrando los datos de cada vehículo. El formato de salida es el siguiente:

=== LISTA DE VEHICULOS ===

Modelo: Toyota Corolla

Año: 2024

Precio: 18500000.0

Estado: DISPONIBLE

Modelo: Fiat Uno

Año: 2015

Precio: 4200000.0

Estado: NO DISPONIBLE

Opción 6 - Salir:

El sistema termina la ejecución de forma limpia, sin errores. El ciclo del menú se detiene y el programa finaliza con un mensaje de despedida:

"Gracias por usar el sistema. Vuelva Pronto"